

Invoice Audit Exercise

Yousef Filmban · Repository: <https://github.com/YSFWRLD/InsuranceAuditTask> · Predictions: `submission.csv` (attached; `outputs/submission.csv` in the repository)

1. Approach and how I measured results

Sequencing. I worked one hospital at a time and kept each contract's logic separate.

- **Hospital 1 first**, because it is the only hospital with labels. The core rules were developed there: conservative matching, a blank total rather than a guessed one, and evidence-based confidence.
- **Hospital 2 next**, to test the approach on a contract written in prose, with conservative semantic matching.
- **Hospital 4 next**, as a more structured, table-driven contract.
- **Hospital 5 next**, to improve service-identity coverage while keeping uncertainty explicit.
- **Hospital 3 last**: a contract split across three documents, with an amendment that changes rates part-way through the term.

The submission covers all four scored hospitals: H2 (1,125 invoices), H3 (932), H4 (835) and H5 (1,050), 3,942 rows. Hospital 1 is not in it.

Hospital 1. Against the labels, across all 913 invoices: 58 true positives, 0 false positives, 855 true negatives, 0 false negatives; 908 of the 909 expected totals offered were exact. The H1 labels were visible to me during development, and a few ambiguous clauses were settled by checking them on the development split. These figures are therefore development (post-hoc) measurements, **not** an untouched estimate of how the method generalises. Details are in `outputs/hospital_1/evaluation_report.md`.

Hospitals 2 to 5. There are no labels, so accuracy cannot be measured directly. Instead I checked:

- **contract consistency**: every rate, multiplier and rule is parsed from the contract text with its clause, and the parser stops on wording it does not recognise;
- **deterministic recalculation**: every proven total is priced step by step from the contract, with a trace for every line;
- **regression tests**: 874 automated tests, including boundary cases for the contract rules and the H3 amendment date;
- **reproducibility**: the full submission regenerates byte-for-byte offline, with no API keys;
- **coverage**: how often an expected total could actually be reconstructed. H2 237 of 1,125, H3 848 of 932, H4 307 of 835 and H5 939 of 1,050 totals are proven; the rest are deliberately blank.

2. Where I was uncertain, and why

My guiding rule was that a blank expected total is better than a confident wrong one. An amount is left blank whenever the service identity or the contract reading does not support a single answer.

H2 semantic limitation. An LLM classifier proposed identities for 69 unclear description groups, and Jev (TypeSafe) verified them. Under my acceptance gate this stage added **no** verified service matches, so most H2 totals are blank.

Service identity in H3 and H5. Descriptions are free-text shorthand. Both hospitals combine deterministic matching with bounded Jev reviews that choose among contracted services when a description leaves out a word. Billed prices and totals are never used to decide which service a line is. The acceptance gate (probability at least 0.90) was fixed before any answer was seen. In H3, Jev adds 818 identified lines and 495 proven totals; one description group scored 0.89 and stays unresolved.

- **ENT is read as Otolaryngologic**, and only where the other words fit a contracted Otolaryngologic service. It is a human-reviewed reading, never a blanket substitution. 233 H5 totals and 319 of H3's 848 depend on it. I accepted it after a specific review: ENT is the standard abbreviation for ear, nose and throat; in H3 it always sits where the specialty goes, never next to another specialty; and several of the same descriptions also appear with "otolaryngologic" spelled out.

- Some description groups stay unresolved because the text genuinely omits a distinguishing word, for example H5's `SUPERVISED SPCM ANLY`. An earlier rule that accepted such cases was removed in my own review as too aggressive.

H3 amendment. The amendment applies by service date, as it says, not by invoice date. Late-2024 services billed on 2025 invoices are therefore priced at the old rates (28 lines, all billed correctly), and 4 lines billed at the wrong period's rate are flagged.

Contract readings. Some clauses admit more than one reading. Each choice, its alternative and, for H3, the measured effect of switching are in the decision logs:

- **Exclusions** ("not billable within N days of"): read as a distance on either side, day N included. This is the least certain reading; all 3 H3 and all 6 H5 exclusion findings depend on it.
- **Cumulative discounts:** a line takes the discount tier already exceeded before it, rather than being split at the threshold.
- **Daily caps:** a breach is flagged, but the total is left blank when the contract does not say what the corrected quantity is.
- **Facility (H5):** the contract refers to the line's facility, but lines carry none, so the invoice's facility is used.

Confidence. The confidence values are evidence-strength judgments, such as whether a finding rests only on the invoice's own fields or on interpreting free text. They are **not** statistically calibrated probabilities.

3. What I would do differently with another week

1. **Get the ambiguous clauses clarified**, above all the direction of the exclusion windows, and run the sensitivity analysis built for H3 on H4 and H5 too.
2. **Calibrate confidence** on held-out or newly labelled data rather than the development labels.
3. **Have the remaining ambiguous service groups reviewed independently.**
4. **Reduce dependence on manual contextual readings** such as ENT, for example by checking them against a small labelled sample of descriptions.
5. **Strengthen tests for contracts that change over time** (amendments and effective dates), and simplify Hospital 2's semantic stage, which did not improve coverage.

I would not raise coverage by lowering the acceptance thresholds. A confident mistake costs more than an explicit uncertainty.

AI assistance and time

- **Development:** code, tests and documentation were written with Claude Code (Claude Opus models) as an AI coding and review assistant. I used it for implementation, code review, contract-reasoning support, and test generation and validation. I set the scope, rules and stopping points, and reviewed the results. All development prompts are versioned in `prompts/`.
- **Runtime:** models are used only as bounded reviewers of service descriptions. H2 uses an LLM classifier and Jev; H3 and H5 use Jev alone, for bounded choices. Pricing, arithmetic and every contract rule are deterministic Python. No model produced ground truth, a price or a total. The runtime prompts and every stored review are versioned in the repository.
- **Time spent:** approximately 4 hours per day over four days.